

Détection de Fake News par Fine-Tuning de RoBERTa

Analyse du biais rédactionnel et pipeline de collecte Bluesky

Auteurs :

Ghiles Menhouk, Wendy Robert, Widad Guertit, Loann Bourdier

Formation :

Master Big Data et Intelligence Artificielle

Année universitaire :

2025 – 2026

18 mai 2026

Résumé

Ce projet porte sur la détection automatique de fake news à l'aide d'un modèle Transformer de type RoBERTa fine-tuné pour une tâche de classification binaire. Une attention particulière a été portée aux biais rédactionnels et au phénomène de *shortcut learning*, fréquemment observés dans les systèmes modernes de détection de désinformation.

Le projet comprend également la conception d'une pipeline ETL de collecte de données depuis le réseau social Bluesky, avec stockage des données dans PostgreSQL.

Les résultats montrent qu'un modèle RoBERTa peut atteindre des performances solides, mais reste vulnérable aux fake news rédigées dans un style journalistique crédible.

Table des matières

Résumé	1
1 Fine-Tuning de RoBERTa pour la Détection de Fake News	4
1.1 Introduction	4
1.2 Contexte scientifique	4
1.2.1 Transformers et modèles de langage	4
1.2.2 Détection de fake news	5
1.2.3 Biais rédactionnel	5
1.3 Présentation du dataset	5
1.3.1 Sources fiables	5
1.3.2 Sources fake news	5
1.3.3 Répartition	6
1.4 Premier entraînement et shortcut learning	6
1.5 Architecture et entraînement	6
1.5.1 Modèle utilisé	6
1.5.2 Tokenization	7
1.5.3 Hyperparamètres	7
1.6 Résultats expérimentaux	7
1.7 Tests de généralisation	7
1.7.1 Style AP avec contenu faux	8
1.7.2 Style Reuters avec contenu absurde	8
1.7.3 Propagande sophistiquée	8
1.8 Limites	8
1.8.1 Absence de fact-checking	8
1.8.2 Biais stylistique résiduel	8
1.8.3 Limitation de longueur	8
1.9 Perspectives	8
2 Pipeline de Collecte de Données Bluesky	10
2.1 Objectif	10
2.2 Préparation de l'environnement	10

2.2.1	Installation de Python	10
2.2.2	Installation des dépendances	10
2.3	PostgreSQL	11
2.3.1	Création de la base	11
2.3.2	Variables d'environnement	11
2.4	Architecture du pipeline	11
2.5	Collecte des données	11
2.6	Pipeline ETL	12
2.7	Exécution	12
2.7.1	Collecte	12
2.7.2	ETL	12
3	Classification et Exécution du Projet	13
3.1	Présentation générale	13
3.2	Méthode 1 : Exécution via Streamlit	13
3.2.1	Fonctionnalités disponibles	13
3.3	Méthode 2 : Exécution manuelle via terminal	14
3.4	Architecture générale de la pipeline	14
3.5	Collecte et traitement des données Bluesky	14
3.6	Lancement du pipeline ETL	15
3.6.1	Collecte des données	15
3.6.2	Nettoyage et transformation ETL	15
3.7	Lancement de la classification	15
3.8	Analyse des émotions	16
3.9	Conclusion	16
	Conclusion Générale	17

1 Fine-Tuning de RoBERTa pour la Détection de Fake News

1.1 Introduction

La détection automatique de fake news constitue aujourd’hui un enjeu majeur en traitement automatique du langage naturel (NLP). Avec l’augmentation massive des contenus générés en ligne, les réseaux sociaux et plateformes numériques sont devenus des vecteurs importants de désinformation.

L’objectif de cette partie du projet est de construire un modèle capable de distinguer des articles journalistiques fiables de contenus de désinformation à l’aide d’un Transformer fine-tuné.

Le modèle choisi est **RoBERTa-base**, une architecture dérivée de BERT optimisée pour le pré-entraînement massif sur de grands corpus textuels.

1.2 Contexte scientifique

1.2.1 Transformers et modèles de langage

Depuis l’introduction des Transformers par Vaswani et al., les architectures basées sur l’attention ont profondément transformé le domaine du NLP.

BERT [1] puis RoBERTa [2] ont démontré que le pré-entraînement auto-supervisé sur de grands corpus permettait d’obtenir des représentations linguistiques extrêmement performantes.

RoBERTa améliore BERT grâce :

- à un pré-entraînement plus long,
- à une meilleure stratégie d’optimisation,
- à des volumes de données plus importants.

1.2.2 Détection de fake news

La détection automatique de fake news est généralement formulée comme une tâche de classification supervisée.

Cependant, plusieurs travaux récents montrent que les modèles modernes ne détectent pas directement la vérité factuelle, mais plutôt des corrélations statistiques présentes dans les données d’entraînement.

1.2.3 Biais rédactionnel

Un problème central de la littérature récente concerne le **biais rédactionnel**.

Cruickshank et al. montrent que les détecteurs de fake news apprennent souvent :

- le style journalistique,
- le ton rédactionnel,
- certains marqueurs lexicaux,
- des signatures de source.

Wu et al. montrent qu’une fake news réécrite dans un style crédible peut fortement perturber les modèles de classification.

Ce phénomène est appelé **shortcut learning** : le modèle apprend des raccourcis statistiques au lieu du véritable signal sémantique.

1.3 Présentation du dataset

Le dataset utilisé est un corpus nommé *MisinfoSuperset*, composé d’environ 79 000 articles journalistiques en anglais.

1.3.1 Sources fiables

Les articles fiables proviennent principalement de :

- Reuters,
- New York Times,
- Washington Post.

1.3.2 Sources fake news

Les fake news proviennent :

- de sites d’extrême droite américains,
- d’EUvsDisinfo,
- du dataset de Ahmed et al.

1.3.3 Répartition

Classe	Nombre d'exemples
TRUE	34 975
FAKE	43 642

TABLE 1.1 – Répartition des labels

Le dataset a été séparé en :

- 70% entraînement,
- 15% validation,
- 15% test.

1.4 Premier entraînement et shortcut learning

Le premier entraînement du modèle a produit des résultats presque parfaits :

- F1-score : 0.9994,
- training loss : 0.056.

Cependant, l'analyse qualitative a montré que le modèle exploitait des signatures éditoriales telles que :

- (Reuters),
- (AP).

Le modèle apprenait donc des indices de source plutôt qu'une véritable représentation de la désinformation.

Cette expérience constitue un exemple classique de shortcut learning.

Afin de corriger ce problème, toutes les signatures éditoriales ont été supprimées avant le second entraînement.

1.5 Architecture et entraînement

1.5.1 Modèle utilisé

Le modèle utilisé est :

- `roberta-base`,
- `RobertaForSequenceClassification`,
- classification binaire.

1.5.2 Tokenization

Le tokenizer utilisé repose sur Byte Pair Encoding (BPE).

Configuration :

- `max_length` = 256,
- troncature activée.

1.5.3 Hyperparamètres

Hyperparamètre	Valeur
Epochs	3
Batch size	16
Gradient accumulation	2
Batch effectif	32
Learning rate	2×10^{-5}
Warmup steps	200
Weight decay	0.01
FP16	Oui
GPU	Tesla T4

TABLE 1.2 – Configuration d’entraînement

1.6 Résultats expérimentaux

Métrique	Valeur
Accuracy	0.9484
Precision	0.9568
Recall	0.9469
F1-score	0.9518

TABLE 1.3 – Résultats finaux sur le test set

Le second entraînement produit des performances solides tout en restant beaucoup plus réaliste que le premier.

1.7 Tests de généralisation

Plusieurs articles artificiels ont été rédigés manuellement afin d’évaluer la robustesse du modèle.

1.7.1 Style AP avec contenu faux

Un faux article rédigé dans un style AP crédible a été classé TRUE avec 94.5% de confiance.

1.7.2 Style Reuters avec contenu absurde

Un article Reuters contenant un contenu volontairement absurde a été classé FAKE avec 63.8% de confiance.

1.7.3 Propagande sophistiquée

Une fake news rédigée dans un style proche de l'AFP a été classée TRUE avec 76.5% de confiance.

Ce test montre que le modèle reste vulnérable aux contenus bien rédigés.

1.8 Limites

1.8.1 Absence de fact-checking

Le modèle ne vérifie pas la réalité des faits.

Il apprend uniquement des régularités statistiques présentes dans les données.

1.8.2 Biais stylistique résiduel

Même après nettoyage du dataset, le modèle reste sensible au style journalistique professionnel.

1.8.3 Limitation de longueur

Le paramètre :

$$max_length = 256$$

entraîne la troncature d'une partie importante des articles.

1.9 Perspectives

Plusieurs améliorations peuvent être envisagées :

- augmentation de `max_length`,
- utilisation de Longformer,

- entraînement adversarial de style,
- ajout de fact-checking externe,
- enrichissement du corpus,
- utilisation de CamemBERT pour le français.

2 Pipeline de Collecte de Données Bluesky

2.1 Objectif

Cette partie du projet concerne la collecte et le stockage de données issues de Bluesky afin de constituer une base exploitable pour des traitements NLP futurs.

L'architecture suit une logique ETL :

- Extract,
- Transform,
- Load.

2.2 Préparation de l'environnement

2.2.1 Installation de Python

Le projet utilise Python 3.11 ou supérieur.

Création d'un environnement virtuel :

```
python -m venv venv
```

Activation :

```
Windows :  
venv\Scripts\activate  
  
Linux/macOS :  
source venv/bin/activate
```

2.2.2 Installation des dépendances

```
pip install requests python-dotenv langdetect psycpg2
```

Bibliothèque	Rôle
requests	Communication API Bluesky
python-dotenv	Variables d'environnement
langdetect	Détection de langue
psycopg2	Connexion PostgreSQL
re	Nettoyage de texte
datetime	Gestion des dates

TABLE 2.1 – Bibliothèques utilisées

2.3 PostgreSQL

2.3.1 Création de la base

```
CREATE DATABASE fake_news_project;
```

2.3.2 Variables d'environnement

```
BLUESKY_HANDLE=votre_identifiant
BLUESKY_PASSWORD=votre_password

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=fake_news_project
POSTGRES_USER=postgres
POSTGRES_PASSWORD=votre_password
```

2.4 Architecture du pipeline

Deux scripts principaux composent la pipeline.

Script	Fonction
collect.py	Collecte des posts
etl _{bluesky} .py	Nettoyage et transformation

TABLE 2.2 – Architecture logicielle

2.5 Collecte des données

Le script `collect.py` :

- se connecte à l'API Bluesky,
- récupère les publications,
- filtre les langues,
- stocke les données dans `raw_posts`.

2.6 Pipeline ETL

Le script `etl_bluesky.py` :

- nettoie les textes,
- prépare les données NLP,
- stocke les résultats dans `processed_posts`.

2.7 Exécution

2.7.1 Collecte

```
python collect.py
```

2.7.2 ETL

```
python etl_bluesky.py
```

3 Classification et Exécution du Projet

3.1 Présentation générale

Le projet peut être exécuté de deux manières distinctes :

- via une interface graphique développée avec Streamlit ;
- via une exécution manuelle en ligne de commande.

Cette double approche permet à la fois :

- une utilisation simplifiée pour l'utilisateur final ;
- une exécution plus technique et modulaire adaptée au développement et au débogage.

3.2 Méthode 1 : Exécution via Streamlit

Une interface graphique a été développée avec Streamlit afin de simplifier l'utilisation du pipeline complet.

Cette interface centralise les principales fonctionnalités du projet et permet à l'utilisateur d'exécuter les différentes étapes sans manipulation directe du terminal.

3.2.1 Fonctionnalités disponibles

L'application Streamlit contient plusieurs boutons permettant de lancer les différentes étapes du pipeline :

- un bouton de collecte et nettoyage des données ;
- un bouton de lancement de la classification des publications ;
- une page dédiée au monitoring Green IT.

Cette approche améliore :

- l'accessibilité du projet ;
- l'expérience utilisateur ;
- la démonstration du pipeline dans un contexte académique ou professionnel.

3.3 Méthode 2 : Exécution manuelle via terminal

Le projet peut également être exécuté manuellement via des scripts Python exécutés dans le terminal.

Cette approche offre :

- un contrôle plus fin des différentes étapes ;
- une meilleure modularité ;
- une exécution possible sur des environnements sans interface graphique.

3.4 Architecture générale de la pipeline

L'architecture du projet repose sur une logique ETL (*Extract – Transform – Load*). Deux scripts principaux composent la pipeline de préparation des données :

Script	Fonction
<code>collect.py</code>	Collecte et stockage brut des publications
<code>etl_bluesky.py</code>	Nettoyage et transformation des données

TABLE 3.1 – Scripts principaux de la pipeline ETL

Cette séparation présente plusieurs avantages :

- une meilleure maintenance du code ;
- une architecture plus scalable ;
- une réutilisation plus simple des données ;
- une meilleure traçabilité des traitements.

3.5 Collecte et traitement des données Bluesky

Le script `collect.py` automatise la récupération des publications depuis Bluesky. Les publications collectées sont stockées dans une table nommée :

raw_posts

Le nettoyage des données est ensuite effectué par le script :

etl_bluesky.py

Cette étape permet de rendre les textes exploitables pour les traitements NLP futurs :

- suppression du bruit textuel ;
- nettoyage des caractères spéciaux ;

- préparation des textes ;
- standardisation des données.

Les données nettoyées sont ensuite stockées dans une seconde table :

processed_posts

3.6 Lancement du pipeline ETL

3.6.1 Collecte des données

Une fois PostgreSQL configuré et le fichier `.env` correctement renseigné, la collecte peut être exécutée avec la commande suivante :

```
python collect.py
```

Le script réalise automatiquement :

- la connexion à Bluesky ;
- la récupération des publications ;
- le filtrage linguistique ;
- le stockage des données brutes.

3.6.2 Nettoyage et transformation ETL

Après la collecte, le pipeline ETL peut être exécuté avec :

```
python etl_bluesky.py
```

Cette étape prépare les données pour les traitements de machine learning et de NLP.

3.7 Lancement de la classification

Une fois les données nettoyées et stockées dans la table `processed_posts`, l'étape de classification peut être lancée.

Cette étape applique le modèle RoBERTa fine-tuné afin d'effectuer la détection de fake news sur les publications collectées.

Commande d'exécution :

```
python classification.py
```

Le script :

- charge les données nettoyées ;
- applique le tokenizer RoBERTa ;

- exécute le modèle de classification ;
- produit les prédictions TRUE / FAKE.

3.8 Analyse des émotions

En complément de la classification des fake news, une analyse émotionnelle des publications peut également être effectuée.

Cette étape permet d'étudier les émotions dominantes présentes dans les publications collectées.

Le lancement s'effectue avec :

```
python emotion.py
```

Cette analyse peut être utilisée pour :

- étudier la propagation émotionnelle de la désinformation ;
- détecter certains patterns émotionnels ;
- enrichir les futures analyses NLP du projet.

3.9 Conclusion

L'architecture du projet permet une exécution flexible grâce à deux approches complémentaires :

- une interface graphique Streamlit adaptée à l'utilisation interactive ;
- une exécution modulaire en ligne de commande adaptée au développement et à l'automatisation.

Cette organisation améliore la maintenabilité du projet et facilite les futures extensions du pipeline NLP.

Conclusion Générale

Ce projet combine deux composantes complémentaires :

- un système de détection de fake news basé sur RoBERTa,
- une infrastructure ETL de collecte de données Bluesky.

Les expériences montrent qu'un modèle Transformer peut obtenir de bonnes performances tout en restant vulnérable aux biais rédactionnels.

Le projet met également en évidence l'importance de la qualité du dataset et les dangers du shortcut learning dans les tâches de classification NLP.

Bibliographie

- [1] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding. 2018.
- [2] Liu, Y. et al. RoBERTa : A Robustly Optimized BERT Pretraining Approach. 2019.
- [3] Wu, Y. et al. Fake News in Sheep's Clothing : Robust Fake News Detection Against LLM-Empowered Style Attacks. 2023.
- [4] Su, J. et al. Fake News Detectors are Biased against Texts Generated by Large Language Models. 2023.
- [5] Cruickshank, I. et al. Writing Style Bias in Satirical News Detection. 2023.